

Hierarchical Codebase Indexing Framework for Large-Scale Repositories and Agentic Software Engineering

Zeroth Review — Project Handout

Department of Computer Science and Engineering · 23CS701 Project-I · Batch 18

Problem

In large repositories (4,000+ files, ~50,000 dependencies), the standard flat dependency graph does not scale: it is too dense to visualise, too large for AI assistants to process within their context window, and inefficient to search. Directory trees do not help, as they reflect file organisation rather than actual code dependencies.

Objective

To automatically transform a flat repository dependency graph into a navigable, multi-level hierarchy — Repository → Groups → Files → Functions — that scales for visualisation, navigation, and retrieval while preserving every dependency relationship.

Technical Approach

1. Dependency extraction. Tree-Sitter parses each source file into an AST; these are stitched into a cross-file directed dependency graph of files, classes, and functions. Each edge is assigned a dependency strength computed from import frequency, method-call frequency, and shared-type usage.
2. Structural-quality assessment. Each region (Phase-1: a Java package) is scored using established package metrics — cohesion (intra-region dependency strength, normalised by size) and coupling (boundary-crossing strength relative to total incident strength). These are mapped to a normalised structural-quality score in the range $[0, 1]$.
3. Adaptive construction (core contribution). Each region's score is compared to a calibrated decision boundary: regions scoring above it are preserved (existing package boundaries kept); regions below are reconstructed via dependency-based community detection (Louvain). All scores, decisions, and parameters are recorded for auditability.
4. Hierarchy assembly & indexing. Regions are assembled into a bounded multi-level tree (cross-group dependencies aggregated and preserved) and serialised to JSON indices — repository.json, hierarchy.json, nodes.json, edges.json, metadata.json — consumable by the visualisation layer, AI agents, and retrieval.

Principal Contribution

Existing tools apply a single global grouping strategy across an entire repository. Because real repositories mix well- and poorly-structured regions, this work introduces adaptive, per-region hierarchy construction: preserve where structure is sound, reconstruct where it is not. The pipeline is deterministic — through content-addressed identifiers and seeded community detection, the same repository always yields an identical hierarchy, making results reproducible and auditable.

Outcomes

- Scalable navigation — only a bounded number of nodes are rendered per level (semantic zoom).
- Reduced retrieval scope for AI assistants — descend to the relevant branch instead of scanning the whole repository.
- Blast-radius analysis — given a modified node, a reverse-dependency traversal returns the impacted files, functions, and hierarchy regions.

Phase-I Plan

Review	Deliverable
First	Dependency-graph extraction (Tree-Sitter → weighted graph)
Second	Adaptive hierarchical construction (assessment → preserve/reconstruct → index)
Third	Interactive multi-level visualisation, evaluated against a flat-graph baseline

Stack: TypeScript / Node.js · Tree-Sitter · graphology · React + React Flow · JSON-based indexing.